RAG-Based Customer Support Assistant

---

1. Introduction

1.1 What is Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a system architecture that combines information retrieval with language model generation. Instead of relying solely on pre-trained knowledge, the system retrieves relevant data from an external knowledge base at runtime and uses it as grounding context for response generation.

This ensures that generated responses are:

- contextually accurate
- traceable to source data
- constrained within known information boundaries

1.2 Why RAG is Required

Standalone language models are insufficient for production support systems due to:

- lack of domain-specific grounding
- inability to reference real documents
- risk of hallucinated or unverifiable answers

RAG mitigates these limitations by introducing a retrieval layer that enforces factual grounding and improves response reliability.

1.3 Use Case Overview

The system is designed for automated customer support where:

- knowledge is stored in structured documents (PDFs)
- users submit natural language queries
- the system retrieves relevant information

- responses are generated strictly from retrieved content
- unresolved or low-confidence cases are escalated to human agents

---

2. System Architecture Explanation

2.1 Architectural Overview

The system operates as two coordinated pipelines:

- **Ingestion Pipeline** → transforms documents into searchable vector representations
- **Query Pipeline** → retrieves relevant context and generates responses

---

2.2 Component Interaction Flow

User → API Layer → Application Service → Embedding → Retrieval → LLM → Decision Engine → Response / Escalation

---

2.3 Component Responsibilities

- **API Layer**
  Handles request validation and response standardization
- **Application Service**
  Orchestrates end-to-end query execution
- **Embedding Module**
  Converts text and queries into vector space
- **Vector Store**
  Stores embeddings and supports similarity search
- **Retrieval Module**
  Identifies relevant document chunks
- **LLM Layer**
  Generates responses constrained by retrieved context

- **Workflow Engine**
  Controls execution and routing logic
- **Escalation System**
  Handles unresolved or uncertain queries

---

3. Design Decisions

3.1 Chunking Strategy

A fixed-size chunking approach with overlap is used.

Rationale:

- ensures contextual continuity across chunk boundaries
- balances retrieval precision with contextual completeness

This avoids fragmentation while maintaining retrievability.

3.2 Embedding Strategy

A single embedding model is used for both:

- document chunks
- user queries

Rationale:

- maintains consistency in vector space representation
- improves semantic similarity accuracy

3.3 Retrieval Approach

Top-K similarity search using cosine similarity is implemented.

Rationale:

- provides efficient semantic matching
- ensures deterministic retrieval behavior

- minimizes computational overhead

3.4 Prompt Design Logic

The prompt is constructed using retrieved chunks as the sole source of truth.

Constraints enforced:

- response must be grounded in provided context
- model must not introduce external knowledge
- structured output is required for downstream processing

---

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph is used to implement a controlled execution workflow rather than a linear pipeline.

It enables:

- state-driven processing
- conditional branching
- modular execution units

4.2 Workflow Structure

START → draft_answer → route_query → finalize_answer | create_escalation → END

4.3 Node Responsibilities

draft_answer

- constructs context
- invokes LLM

- parses structured response

---

route_query

- evaluates confidence metrics
- determines execution path

---

finalize_answer

- returns validated response

---

create_escalation

- persists escalation case
- stores context and reasoning

4.4 State Management

The system maintains a structured state across nodes containing:

- query
- retrieved context
- confidence scores
- generated response
- routing decision

State transitions are deterministic and governed by decision logic.

---

5. Conditional Logic

5.1 Intent Handling

No explicit intent classifier is used.
Intent is inferred through retrieval success and model response confidence.

5.2 Routing Decisions

Automated Response

- sufficient retrieval confidence
- sufficient model confidence
- no escalation signal

Escalation

- absence of relevant context
- low confidence thresholds
- unsupported or ambiguous queries

---

6. Human-in-the-Loop (HITL) Implementation

6.1 Role of Human Intervention

Human intervention acts as a fallback mechanism for cases where automated resolution is unreliable.

6.2 Execution Flow

1. system detects low-confidence condition
2. escalation ticket is created
3. context and draft response are stored
4. human agent reviews and resolves
5. resolution is persisted

6.3 Benefits

- ensures correctness in edge cases

- prevents propagation of incorrect responses
- improves system reliability

6.4 Limitations

- introduces response latency
- requires manual operational effort
- limits full automation

---

7. Challenges & Trade-offs

7.1 Retrieval Accuracy vs Latency

Increasing retrieval depth improves recall but increases response time and prompt size.

7.2 Chunk Size vs Context Quality

- larger chunks preserve context but reduce precision
- smaller chunks improve precision but risk context loss

7.3 Cost vs Performance

Higher-quality responses require larger prompts and more compute, increasing cost.

---

8. Testing Strategy

8.1 Testing Approach

- unit testing for individual modules
- integration testing across pipelines

- scenario-based manual validation

8.2 Sample Queries

- refund and policy-related queries
- delivery and logistics queries
- modification and exception scenarios
- unsupported or ambiguous queries

8.3 Validation Criteria

- correctness of retrieval
- grounding of responses
- accuracy of routing decisions

---

9. Future Enhancements

9.1 Multi-Document Support

Enable indexing and retrieval across multiple knowledge sources.

9.2 Feedback Loop Integration

Incorporate resolved escalations into knowledge base improvement.

9.3 Memory Integration

Support session-based context for multi-turn interactions.

9.4 Deployment Enhancements

- containerized deployment

- scalable API infrastructure
- monitoring and observability

Conclusion

This system implements a controlled RAG architecture that prioritizes:

- grounded response generation
- deterministic workflow execution
- explicit decision-making logic
- safe fallback through human escalation

It is designed to deliver reliable and explainable customer support automation in a production-oriented environment.